

In HIRE-ASSISTANT, assuming that the candidates are presented in a random order, what is the probability that you hire exactly twice?

5.2-3

Use indicator random variables to compute the expected value of the sum of n dice.

5.2-4

This exercise asks you to (partly) verify that linearity of expectation holds even if the random variables are not independent. Consider two 6-sided dice that are rolled independently. What is the expected value of the sum? Now consider the case where the first die is rolled normally and then the second die is set equal to the value shown on the first die. What is the expected value of the sum? Now consider the case where the first die is rolled normally and the second die is set equal to 7 minus the value of the first die. What is the expected value of the sum?

5.2-5

Use indicator random variables to solve the following problem, which is known as the *hat-check problem*. Each of n customers gives a hat to a hat-check person at a restaurant. The hat-check person gives the hats back to the customers in a random order. What is the expected number of customers who get back their own hat?

5.2-6

Let $A[1 : n]$ be an array of n distinct numbers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an *inversion* of A . (See Problem 2-4 on page 47 for more on inversions.) Suppose that the elements of A form a uniform random permutation of $\langle 1, 2, \dots, n \rangle$. Use indicator random variables to compute the expected number of inversions.

5.3 Randomized algorithms

In the previous section, we showed how knowing a distribution on the inputs can help us to analyze the average-case behavior of an algorithm. What if you do not know the distribution? Then you cannot

perform an average-case analysis. As mentioned in Section 5.1, however, you might be able to use a randomized algorithm.

For a problem such as the hiring problem, in which it is helpful to assume that all permutations of the input are equally likely, a probabilistic analysis can guide us when developing a randomized algorithm. Instead of *assuming* a distribution of inputs, we *impose* a distribution. In particular, before running the algorithm, let's randomly permute the candidates in order to enforce the property that every permutation is equally likely. Although we have modified the algorithm, we still expect to hire a new office assistant approximately $\ln n$ times. But now we expect this to be the case for *any* input, rather than for inputs drawn from a particular distribution.

Let us further explore the distinction between probabilistic analysis and randomized algorithms. In Section 5.2, we claimed that, assuming that the candidates arrive in a random order, the expected number of times you hire a new office assistant is about $\ln n$. This algorithm is deterministic: for any particular input, the number of times a new office assistant is hired is always the same. Furthermore, the number of times you hire a new office assistant differs for different inputs, and it depends on the ranks of the various candidates. Since this number depends only on the ranks of the candidates, to represent a particular input, we can just list, in order, the ranks $\langle \text{rank}(1), \text{rank}(2), \dots, \text{rank}(n) \rangle$ of the candidates. Given the rank list $A_1 = \langle 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 \rangle$, a new office assistant is always hired 10 times, since each successive candidate is better than the previous one, and lines 5–6 of HIRE-ASSISTANT are executed in each iteration. Given the list of ranks $A_2 = \langle 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 \rangle$, a new office assistant is hired only once, in the first iteration. Given a list of ranks $A_3 = \langle 5, 2, 1, 8, 4, 7, 10, 9, 3, 6 \rangle$, a new office assistant is hired three times, upon interviewing the candidates with ranks 5, 8, and 10. Recalling that the cost of our algorithm depends on how many times you hire a new office assistant, we see that there are expensive inputs such as A_1 , inexpensive inputs such as A_2 , and moderately expensive inputs such as A_3 .

Consider, on the other hand, the randomized algorithm that first permutes the list of candidates and then determines the best candidate. In this case, we randomize in the algorithm, not in the input distribution. Given a particular input, say A_3 above, we cannot say how many times the maximum is updated, because this quantity differs with each run of the algorithm. The first time you run the algorithm on A_3 , it might produce the permutation A_1 and perform 10 updates. But the second time you run the algorithm, it might produce the permutation A_2 and perform only one update. The third time you run the algorithm, it might perform some other number of updates. Each time you run the algorithm, its execution depends on the random choices made and is likely to differ from the previous execution of the algorithm. For this algorithm and many other randomized algorithms, *no particular input elicits its worst-case behavior*. Even your worst enemy cannot produce a bad input array, since the random permutation makes the input order irrelevant. The randomized algorithm performs badly only if the random-number generator produces an “unlucky” permutation.

For the hiring problem, the only change needed in the code is to randomly permute the array, as done in the RANDOMIZED-HIRE-ASSISTANT procedure. This simple change creates a randomized algorithm whose performance matches that obtained by assuming that the candidates were presented in a random order.

RANDOMIZED-HIRE-ASSISTANT(n)

```
1 randomly permute the list
  of candidates
2 HIRE-ASSISTANT( $n$ )
```

Lemma 5.3

The expected hiring cost of the procedure RANDOMIZED-HIRE-ASSISTANT is $O(c_h \ln n)$.

Proof Permuting the input array achieves a situation identical to that of the probabilistic analysis of HIRE-ASSISTANT in Section 5.2.



By carefully comparing Lemmas 5.2 and 5.3, you can see the difference between probabilistic analysis and randomized algorithms. Lemma 5.2 makes an assumption about the input. Lemma 5.3 makes no such assumption, although randomizing the input takes some additional time. To remain consistent with our terminology, we couched Lemma 5.2 in terms of the average-case hiring cost and Lemma 5.3 in terms of the expected hiring cost. In the remainder of this section, we discuss some issues involved in randomly permuting inputs.

Randomly permuting arrays

Many randomized algorithms randomize the input by permuting a given input array. We'll see elsewhere in this book other ways to randomize an algorithm, but now, let's see how we can randomly permute an array of n elements. The goal is to produce a *uniform random permutation*, that is, a permutation that is as likely as any other permutation. Since there are $n!$ possible permutations, we want the probability that any particular permutation is produced to be $1/n!$.

You might think that to prove that a permutation is a uniform random permutation, it suffices to show that, for each element $A[i]$, the probability that the element winds up in position j is $1/n$. Exercise 5.3-4 shows that this weaker condition is, in fact, insufficient.

Our method to generate a random permutation permutes the array *in place*: at most a constant number of elements of the input array are ever stored outside the array. The procedure RANDOMLY-PERMUTE permutes an array $A[1 : n]$ in place in $\Theta(n)$ time. In its i th iteration, it chooses the element $A[i]$ randomly from among elements $A[i]$ through $A[n]$. After the i th iteration, $A[i]$ is never altered.

RANDOMLY-PERMUTE(A, n)

```
1 for  $i = 1$  to  $n$ 
2   swap  $A[i]$  with  $A[\text{RANDOM}(i, n)]$ 
```

We use a loop invariant to show that procedure RANDOMLY-PERMUTE produces a uniform random permutation. A *k-permutation* on a set of n elements is a sequence containing k of the n elements, with no repetitions. (See page 1180 in Appendix C.) There are $n!/(n - k)!$ such possible k -permutations.

Lemma 5.4

Procedure RANDOMLY-PERMUTE computes a uniform random permutation.

Proof We use the following loop invariant:

Just prior to the i th iteration of the **for** loop of lines 1–2, for each possible $(i - 1)$ -permutation of the n elements, the subarray $A[1 : i - 1]$ contains this $(i - 1)$ -permutation with probability $(n - i + 1)!/n!$.

We need to show that this invariant is true prior to the first loop iteration, that each iteration of the loop maintains the invariant, that the loop terminates, and that the invariant provides a useful property to show correctness when the loop terminates.

Initialization: Consider the situation just before the first loop iteration, so that $i = 1$. The loop invariant says that for each possible 0-permutation, the subarray $A[1 : 0]$ contains this 0-permutation with probability $(n - i + 1)!/n! = n!/n! = 1$. The subarray $A[1 : 0]$ is an empty subarray, and a 0-permutation has no elements. Thus, $A[1 : 0]$ contains any 0-permutation with probability 1, and the loop invariant holds prior to the first iteration.

Maintenance: By the loop invariant, we assume that just before the i th iteration, each possible $(i - 1)$ -permutation appears in the subarray $A[1 : i - 1]$ with probability $(n - i + 1)!/n!$. We shall show that after the i th iteration, each possible i -permutation appears in the subarray $A[1 : i]$ with probability $(n - i)!/n!$. Incrementing i for the next iteration then maintains the loop invariant.

Let us examine the i th iteration. Consider a particular i -permutation, and denote the elements in it by $\langle x_1, x_2, \dots, x_i \rangle$. This permutation consists of an $(i-1)$ -permutation $\langle x_1, \dots, x_{i-1} \rangle$ followed by the value x_i that the algorithm places in $A[i]$. Let E_1 denote the event in which the first $i-1$ iterations have created the particular $(i-1)$ -permutation $\langle x_1, \dots, x_{i-1} \rangle$ in $A[1 : i-1]$. By the loop invariant, $\Pr \{E_1\} = (n-i+1)!/n!$. Let E_2 be the event that the i th iteration puts x_i in position $A[i]$. The i -permutation $\langle x_1, \dots, x_i \rangle$ appears in $A[1 : i]$ precisely when both E_1 and E_2 occur, and so we wish to compute $\Pr \{E_2 \cap E_1\}$. Using equation (C.16) on page 1187, we have

$$\Pr \{E_2 \cap E_1\} = \Pr \{E_2 \mid E_1\} \Pr \{E_1\}.$$

The probability $\Pr \{E_2 \mid E_1\}$ equals $1/(n-i+1)$ because in line 2 the algorithm chooses x_i randomly from the $n-i+1$ values in positions $A[i : n]$. Thus, we have

$$\begin{aligned} \Pr \{E_2 \cap E_1\} &= \Pr \{E_2 \mid E_1\} \Pr \{E_1\} \\ &= \frac{1}{n-i+1} \cdot \frac{(n-i+1)!}{n!} \\ &= \frac{(n-i)!}{n!}. \end{aligned}$$

Termination: The loop terminates, since it is a **for** loop iterating n times. At termination, $i = n+1$, and we have that the subarray $A[1 : n]$ is a given n -permutation with probability $(n - (n+1) + 1)!/n! = 0!/n! = 1/n!$. Thus, RANDOMLY-PERMUTE produces a uniform random permutation. ■

A randomized algorithm is often the simplest and most efficient way to solve a problem.

Exercises

5.3-1

Professor Marceau objects to the loop invariant used in the proof of Lemma 5.4. He questions whether it holds prior to the first iteration. He reasons that we could just as easily declare that an empty subarray contains no 0-permutations. Therefore, the probability that an empty subarray contains a 0-permutation should be 0, thus invalidating the loop invariant prior to the first iteration. Rewrite the procedure RANDOMLY-PERMUTE so that its associated loop invariant applies to a nonempty subarray prior to the first iteration, and modify the proof of Lemma 5.4 for your procedure.

5.3-2

Professor Kelp decides to write a procedure that produces at random any permutation except the *identity permutation*, in which every element ends up where it started. He proposes the procedure PERMUTE-WITHOUT-IDENTITY. Does this procedure do what Professor Kelp intends?

```
PERMUTE-WITHOUT-IDENTITY( $A, n$ )
```

```
1 for  $i = 1$  to  $n - 1$   
2   swap  $A[i]$  with  $A[\text{RANDOM}(i + 1, n)]$ 
```

5.3-3

Consider the PERMUTE-WITH-ALL procedure on the facing page, which instead of swapping element $A[i]$ with a random element from the subarray $A[i : n]$, swaps it with a random element from anywhere in the array. Does PERMUTE-WITH-ALL produce a uniform random permutation? Why or why not?

```
PERMUTE-WITH-ALL( $A, n$ )
```

```
1 for  $i = 1$  to  $n$   
2   swap  $A[i]$  with  $A[\text{RANDOM}(1, n)]$ 
```

5.3-4

Professor Knieval suggests the procedure PERMUTE-BY-CYCLE to generate a uniform random permutation. Show that each element $A[i]$

has a $1/n$ probability of winding up in any particular position in B . Then show that Professor Knieval is mistaken by showing that the resulting permutation is not uniformly random.

PERMUTE-BY-CYCLE(A, n)

```
1 let  $B[1 : n]$  be a new array
2  $offset = \text{RANDOM}(1, n)$ 
3 for  $i = 1$  to  $n$ 
4    $dest = i + offset$ 
5   if  $dest > n$ 
6      $dest = dest - n$ 
7    $B[dest] = A[i]$ 
8 return  $B$ 
```

5.3-5

Professor Gallup wants to create a *random sample* of the set $\{1, 2, 3, \dots, n\}$, that is, an m -element subset S , where $0 \leq m \leq n$, such that each m -subset is equally likely to be created. One way is to set $A[i] = i$, for $i = 1, 2, 3, \dots, n$, call $\text{RANDOMLY-PERMUTE}(A)$, and then take just the first m array elements. This method makes n calls to the RANDOM procedure. In Professor Gallup's application, n is much larger than m , and so the professor wants to create a random sample with fewer calls to RANDOM .

RANDOM-SAMPLE(m, n)

```
1  $S = \emptyset$ 
2 for  $k = n - m + 1$  to  $n$                                 // iterates  $m$  times
3    $i = \text{RANDOM}(1, k)$ 
4   if  $i \in S$ 
5      $S = S \cup \{k\}$ 
6   else  $S = S \cup \{i\}$ 
7 return  $S$ 
```


Show that the procedure RANDOM-SAMPLE on the previous page returns a random m -subset S of $\{1, 2, 3, \dots, n\}$, in which each m -subset is equally likely, while making only m calls to RANDOM.

★ 5.4 Probabilistic analysis and further uses of indicator random variables

This advanced section further illustrates probabilistic analysis by way of four examples. The first determines the probability that in a room of k people, two of them share the same birthday. The second example examines what happens when randomly tossing balls into bins. The third investigates “streaks” of consecutive heads when flipping coins. The final example analyzes a variant of the hiring problem in which you have to make decisions without actually interviewing all the candidates.

5.4.1 The birthday paradox

Our first example is the *birthday paradox*. How many people must there be in a room before there is a 50% chance that two of them were born on the same day of the year? The answer is surprisingly few. The paradox is that it is in fact far fewer than the number of days in a year, or even half the number of days in a year, as we shall see.

To answer this question, we index the people in the room with the integers $1, 2, \dots, k$, where k is the number of people in the room. We ignore the issue of leap years and assume that all years have $n = 365$ days. For $i = 1, 2, \dots, k$, let b_i be the day of the year on which person i 's birthday falls, where $1 \leq b_i \leq n$. We also assume that birthdays are uniformly distributed across the n days of the year, so that $\Pr \{b_i = r\} = 1/n$ for $i = 1, 2, \dots, k$ and $r = 1, 2, \dots, n$.

The probability that two given people, say i and j , have matching birthdays depends on whether the random selection of birthdays is independent. We assume from now on that birthdays are independent, so that the probability that i 's birthday and j 's birthday both fall on day r is